Decisiones de diseño e implementación del código

Presto — documento técnico para la instancia post-exposición

> **Estado del proyecto:** demo pública con dinero simulado. **Repositorio:** `jrweiss93/PRESTO-MVP` (rama `main`).
> **Backend:** Python 3.14 + Django 6 + Django REST Framework. **Frontend:** React 18 + TypeScript + Vite + Tailwind.
> **Base de datos:** PostgreSQL 16. **Cola:** Redis + Celery. **Última actualización:** noviembre de 2026.

Este documento explica **por qué el código está armado así**, no solamente qué archivos existen. La idea es poder defender las decisiones técnicas después de la exposición: elección del lenguaje, organización en apps, uso de programación orientada a objetos, separación entre modelos y servicios, reglas de dinero, seguridad, frontend, pruebas y límites de la demo.

Para el detalle archivo por archivo están `architecture.md` y `implementacion.md`. Para las decisiones puntuales con contexto y consecuencias están los ADRs. Este texto es la versión narrativa que conecta todo.

Contenido

1. Qué se priorizó al escribir el código
2. Elección del stack
3. Arquitectura general: monolito modular
4. ¿Se usó programación orientada a objetos?
5. Las decisiones críticas sobre el dinero
6. Modelo de datos en una vista rápida
7. Diseño de la API
8. Frontend: decisiones de estructura
9. Pruebas y calidad
10. Seguridad, privacidad y modo demo
11. Infraestructura y despliegue
12. Trade-offs: qué se ganó y qué se dejó pendiente
13. Mapa rápido del código
14. Cómo explicarlo en dos minutos
15. Glosario mínimo

1. Qué se priorizó al escribir el código

Presto no es un ejercicio de "usar todas las tecnologías posibles". Es una demo funcional de una fintech, con una restricción muy fuerte: **el dinero tiene que ser consistente aunque sea simulado**. A partir de eso se priorizó:

1. **Integridad antes que velocidad de desarrollo.** Un préstamo, una cuota o una inversión no pueden quedar a medias. Las operaciones que mueven dinero son atómicas y pasan por un libro mayor.
2. **Claridad de las reglas de negocio.** Las reglas viven en un lugar identificable (`services.py`), no repartidas entre vistas, serializers y modelos.

3. **Trazabilidad.** Cada movimiento financiero tiene quién lo hizo, cuándo, con qué clave de idempotencia y qué asientos generó. Las condiciones de un préstamo quedan congeladas en un snapshot.
4. **Una demo que se pueda levantar y explicar.** Un solo backend, un solo frontend, un solo despliegue y datos de ejemplo reproducibles con `seed_demo`.
5. **Bajo costo operativo.** Render free + Vercel para la demo, con límites documentados y sin dependencias innecesarias.

El resultado es un **monolito modular**: no hay microservicios, ni API gateway, ni múltiples bases de datos. Esa decisión se explica en la sección 3.

---

2. Elección del stack

Pieza	Tecnología	Por qué se eligió	Costo / límite asumido
Lenguaje backend	Python 3.14	Legibilidad, ecosistema maduro, excelente para reglas de negocio y pruebas	Menos rendimiento crudo que otros lenguajes; no es el cuello de botella para esta escala
Framework backend	Django 6	ORM, migraciones, autenticación, admin, validaciones y seguridad "de fábrica"	Framework grande; se compensa con una estructura modular estricta
API	Django REST Framework	Serializers, permisos, paginación, throttling y esquema OpenAPI	Agrega una capa de abstracción sobre Django
Autenticación	SimpleJWT	Access token corto + refresh rotativo + blacklist	Hay que cuidar el almacenamiento del token en el front
Documentación de API	drf-spectacular	El contrato OpenAPI se genera desde el código y se valida en CI	Obliga a mantener anotados los endpoints
Base de datos	PostgreSQL 16	Transacciones, constraints, triggers y bloqueos reales	No se usa SQLite ni siquiera en tests: el comportamiento tiene que ser el de producción
Cola / scheduler	Redis 7 + Celery	Emails, vencimientos y liquidaciones fuera del request	En Render free no hay workers; se documenta como límite
Frontend	React 18 + Vite	SPA rápida, componentes reutilizables, build simple	No hay SSR; la landing pública es estática
Tipado frontend	TypeScript 5.6	Detecta errores de contrato antes de ejecutar	Hay que regenerar tipos cuando cambia la API
Estilos	Tailwind CSS 4	Consistencia visual, dark mode y responsive sin CSS disperso	Requiere disciplina con clases semánticas
Tests backend	pytest + pytest-django + factory_boy	Tests rápidos y expresivos contra PostgreSQL real	La suite completa tarda más que con SQLite
Calidad	ruff + import-linter	Formato, lint y límites entre apps verificados automáticamente	Import-linter obliga a pensar las dependencias
Infra	Docker, Render, Vercel, GitHub Actions, GHCR	Deploy reproducible y gratuito para la demo	Servicios free se duermen y la base expira

¿Por qué Python y no Node/Java/Go?

Porque el problema central no es de throughput, sino de **modelado de reglas financieras y velocidad de entrega**. Django ofrece en un solo paquete:

- un ORM con transacciones y migraciones;
- un sistema de usuarios y permisos;
- un admin para operar KYC y productos;
- validaciones y protección CSRF;
- una comunidad enorme de testing.

Con Node o Go habría que integrar varias piezas para llegar al mismo punto. Para un equipo chico y una demo con fecha de entrega, Django reduce la superficie de código propio y deja más tiempo para las reglas de negocio.

¿Por qué no FastAPI?

FastAPI es una excelente opción para APIs asincrónicas y servicios pequeños. Acá pesaron más el ORM, el admin, la autenticación y las migraciones integradas. Las operaciones financieras de Presto son **transaccionales y secuenciales**, no un caso de alto I/O concurrente; el modelo sync de Django con PostgreSQL es suficiente y más simple de razonar.

¿Por qué no microservicios?

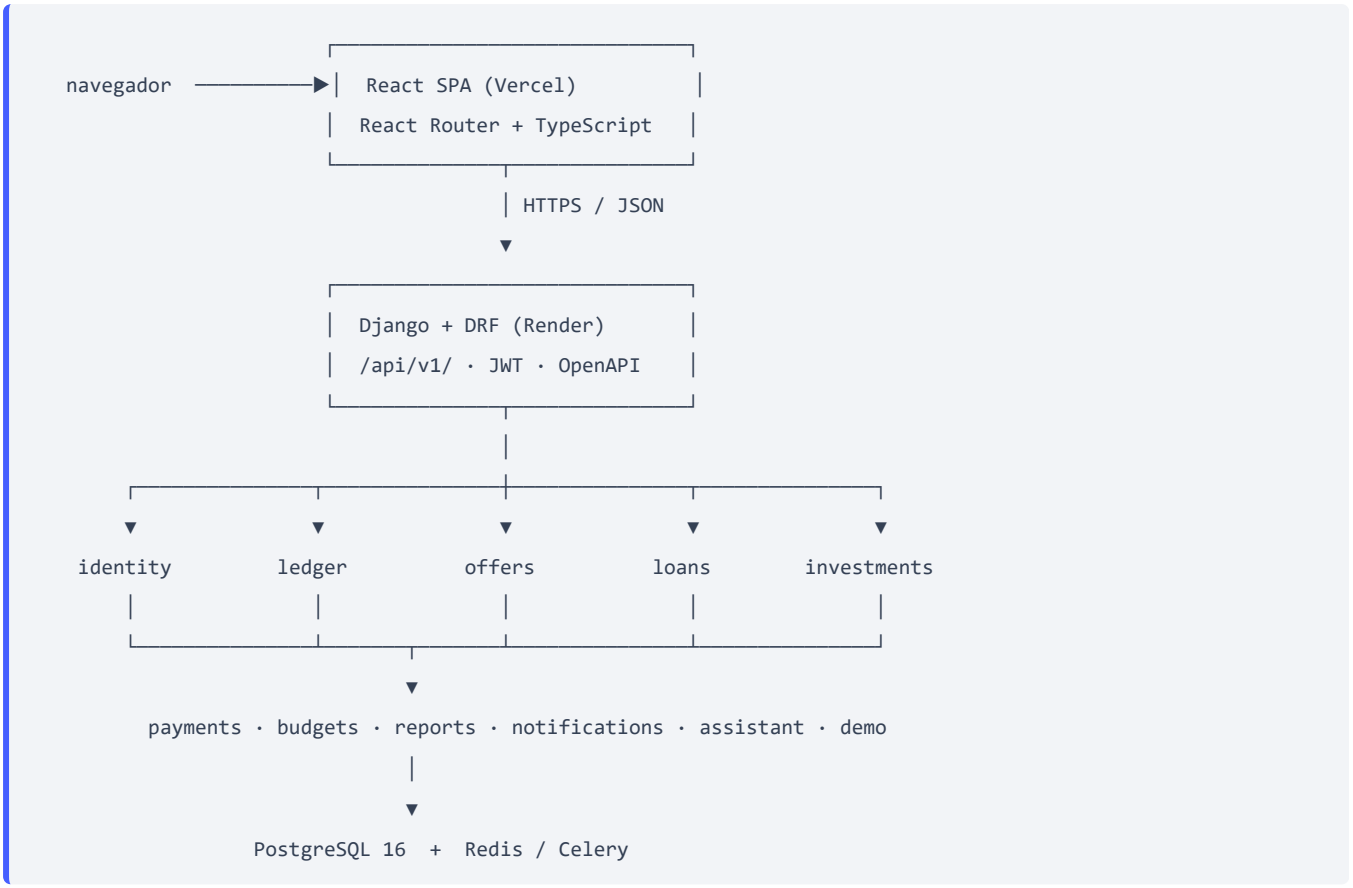
Porque separar identidad, ledger, préstamos e inversiones en servicios independientes habría agregado:

- comunicación entre servicios;
- consistencia distribuida;
- más despliegues y más observabilidad;
- más lugares donde un fallo puede dejar dinero a medias.

Con un monolito modular se mantienen los límites lógicos (apps y contratos de importación) sin pagar el costo operativo de una arquitectura distribuida. Si el producto crece, los límites ya están preparados para extraer una app a un servicio.

---

3. Arquitectura general: monolito modular



3.1 Apps de Django con responsabilidad única

App	Responsabilidad	Modelos principales
`core`	Piezas transversales: modelo base, dinero, errores, paginación, health, meta	`BaseModel` (abstracto)
`identity`	Usuarios, login, perfil, KYC, biometría simulada, cuentas de respaldo	`User`, `Profile`, `PrestoProfile`, `KYCVerification`, `BiometricCheck`, `BackupAccount`
`ledger`	Libro mayor de doble entrada: cuentas, transacciones, asientos y reversos	`Account`, `LedgerTransaction`, `LedgerEntry`
`payments`	Intención de pago, proveedor y webhooks	`Payment`, `WebhookEvent`
`offers`	Ofertas publicadas por prestamistas	`Offer`
`matching`	Búsqueda y comparación de ofertas (solo lectura)	—
`loans`	Préstamos, snapshot de condiciones y cuotas	`Loan`, `OfferTermsSnapshot`, `Installment`
`investments`	Productos de plazo fijo y posiciones	`InvestmentProduct`, `Position`
`budgets`	Categorías, presupuestos y clasificación de gastos	`Category`, `Expense`, `Budget`
`reports`	Reportes de solo lectura y exportación CSV	—
`notifications`	Avisos y preferencias de aviso	`Notification`, `NotificationPreference`
`assistant`	Presti: mercado, noticias, marco legal y chat por reglas	—

App	Responsabilidad	Modelos principales
`demo`	Siembra idempotente del escenario de demostración	—

3.2 Regla de oro: entre apps solo se entra por servicios y selectores

Una app **no importa modelos, vistas ni serializers de otra app**. Solo puede usar:

- `services.py`: operaciones que escriben y aplican reglas;
- `selectors.py`: consultas de lectura y agregados.

Ejemplo: `loans` no importa `Account` del ledger. Llama a `ledger.services.transfer_between_users(...)`. `budgets` no guarda el importe de un gasto: lo deriva del ledger mediante un selector.

Esta regla se verifica con **32 contratos de import-linter** en CI. Si alguien rompe el límite, el build falla. Es la forma de mantener un monolito ordenado sin convertirlo en una bola de dependencias.

3.3 Sin ForeignKey entre apps de dominio

Las referencias cruzadas se guardan como `UUIDField` (`user_id`, `offer_id`, `ledger_transaction_id`, etc.) y se resuelven con el selector de la app dueña. Las ForeignKey se reservan para relaciones internas de una misma app, por ejemplo `Position.product` o `Budget.category`.

Ventaja: las apps quedan desacopladas y se pueden extraer a un servicio sin migrar claves foráneas.

Costo: la base no garantiza la integridad referencial entre apps. Se compensa con servicios, selectores, tests y el hecho de que los UUID no se editan desde el cliente.

3.4 Capas dentro de cada app

```
urls.py ──▶ views.py ──▶ serializers.py     (entrada, formato, permisos)
                │
                ├──▶ services.py           (escritura, reglas, atomicidad)
                └──▶ selectors.py          (lectura, filtros, agregados)
                        │
                        ▼
                    models.py              (campos, constraints, métodos simples)
```

La regla es explícita:

- **Vista fina:** autentica, valida, llama a un service o selector y responde.
- **Service:** contiene la regla de negocio y el `transaction.atomic()`.
- **Selector:** contiene consultas de lectura, sin efectos secundarios.
- **Modelo:** define campos, constraints y comportamientos triviales; no dispara operaciones financieras.
- **Serializer:** valida y formatea; no mueve dinero.

Esta separación es la decisión estructural más importante del backend.

---

4. ¿Se usó programación orientada a objetos?

Sí, pero de forma **pragmática y selectiva**. Python permite mezclar estilos y Presto lo hace a propósito: hay clases donde una clase aporta claridad y hay funciones donde una clase solo agregaría ceremonia.

Para entenderlo sin vueltas, una analogía:

> Una **clase** es el plano de una casa. Un **objeto** es una casa construida con ese plano. Los **atributos** son las habitaciones (monto, tasa, plazo). Los **métodos** son las cosas que la casa sabe hacer. La **herencia** es partir de un plano base. El **encapsulamiento** es no dejar que cualquiera toque la instalación eléctrica. La **abstracción** es pedir "un enchufe" sin importar la marca del aparato. El **polimorfismo** es que el mismo enchufe funcione con una lámpara o con una heladera.

4.1 Los conceptos de POO aplicados a Presto

Concepto	Explicación simple	Ejemplo concreto en Presto
Clase	El molde o plano para crear algo	`Loan` es el molde de un préstamo
Objeto	Una cosa concreta creada con ese molde	El préstamo de Ana por $120.000 a 6 meses
Atributo	Un dato que guarda el objeto	`principal`, `interest_rate`, `term_months`
Método	Una acción que sabe hacer el objeto	`__str__()` devuelve un texto legible para logs
Herencia	Reutilizar un molde base	`Loan` hereda de `BaseModel` el UUID y las fechas
Encapsulamiento	Esconder los detalles y exponer solo lo necesario	El saldo solo lo modifica `ledger.services`; una vista no toca `Account.balance`
Abstracción	Mostrar el "qué" y ocultar el "cómo"	`PaymentProvider` dice "creá un depósito", no cómo lo hace cada proveedor
Polimorfismo	El mismo mensaje produce comportamientos distintos	`provider.create_deposit()` funciona con el proveedor simulado o con Mercado Pago

4.2 Modelos: clases que representan entidades

Cada modelo es una clase que hereda de `models.Model` y representa una tabla y una entidad del negocio. Además de los campos, las clases usan:

- `BaseModel` **abstracto** para todas las entidades: UUID como PK, `created_at` y `updated_at`. Evita repetir tres campos en cada modelo.
- `TextChoices` para estados y tipos: `Loan.Status`, `Installment.Status`, `Account.Kind`, `BackupAccount.Provider`, etc. Son enums tipados que Django valida y que el admin muestra con etiquetas legibles.
- `Meta.constraints` para reglas que la base debe garantizar: montos positivos, monedas válidas, una sola cuenta por usuario y moneda, una sola KYC pendiente por usuario, una sola preferencia por evento.
- **Métodos triviales** como `__str__` para logs y admin, y propiedades como `User.email_verified`.
- **Managers** como `UserManager`, que encapsula la creación de usuarios con email en lugar de `username`.

Ejemplos concretos:

Clase	Rol	Detalle interesante
`BaseModel`	Entidad base abstracta	UUID, timestamps; no crea tabla propia
`User`	Usuario del sistema	Hereda de `AbstractUser`, elimina `username` y usa email único

Clase	Rol	Detalle interesante
`Profile`	Datos personales	Uno a uno con `User`
`PrestoProfile`	Perfil operativo	`borrower`, `lender` o `investor`; se crea uno activo al registrarse
`KYCVerification`	Solicitud de identidad	Historial de estados `pending/approved/rejected`
`BiometricCheck`	Validación biométrica simulada	Guarda puntajes y un hash de referencia, nunca la imagen
`BackupAccount`	Cuenta de respaldo de repago	Guarda proveedor, referencia enmascarada y saldo declarado; no credenciales
`Account`	Cuenta contable	Billetera de usuario, custodia o sistema; saldo materializado
`LedgerTransaction`	Movimiento contable	Agrupa asientos y guarda idempotencia
`LedgerEntry`	Asiento	Débito o crédito sobre una cuenta
`Offer`	Oferta de préstamo	Tasa, plazo, monto y requisitos
`Loan`	Préstamo aceptado	Nace `active`; estados `active/paid/defaulted`
`OfferTermsSnapshot`	Condiciones congeladas	JSON inmutable de la oferta aceptada
`Installment`	Cuota	Capital, interés, vencimiento y estado
`InvestmentProduct`	Producto de plazo fijo	Lo crea solo el staff desde el admin
`Position`	Inversión abierta	Monto, vencimiento e interés calculado
`Category`, `Expense`, `Budget`	Presupuestos	`Expense` no guarda importe: lo deriva del ledger
`Payment`, `WebhookEvent`	Pagos	Intención, estado y evento externo idempotente
`Notification`, `NotificationPreference`	Avisos	Estado de envío y preferencia por evento

La herencia se usa **solo donde el framework la exige o donde evita duplicación real**: `BaseModel`, `AbstractUser`, `BasePermission`, `ModelSerializer`, `abc.ABC`. No hay jerarquías profundas ni herencia múltiple para las reglas de negocio.

Una aclaración sobre los perfiles: el registro crea **un** `PrestoProfile` activo con el tipo elegido. El servicio `set_profile` se usa para fijar o corregir el perfil de las cuentas de demo y elimina cualquier otro perfil del usuario; la API no expone un cambio de perfil. Los permisos siempre leen el perfil activo, no un dato que el cliente pueda mandar en cada request.

4.3 Cómo se lee una clase de Django (comentada)

Este es el modelo `Loan`, simplificado y con comentarios. La idea es mostrar qué representa cada bloque, como si estuviéramos explicando la clase en voz alta:

```
class Loan(BaseModel):
    # HERENCIA: BaseModel ya aporta id (UUID), created_at y updated_at.
    # No repetimos esos tres campos en cada tabla del sistema.

    class Status(models.TextChoices):
        # ENUMERACIÓN: los únicos estados válidos de un préstamo.
        # Django los valida y el admin muestra la etiqueta de la derecha.
        ACTIVE = 'active', 'Activo'
        PAID = 'paid', 'Pagado'
        DEFAULTED = 'defaulted', 'En mora'

    # ATRIBUTOS: cada uno es una columna de la tabla loans_loan.
    # Las referencias a otras apps son UUID, no ForeignKey cruzadas.
    borrower_user_id = models.UUIDField()
    lender_user_id = models.UUIDField()
    offer_id = models.UUIDField()

    # El dinero siempre es Decimal, nunca float.
    principal = models.DecimalField(max_digits=18, decimal_places=2)
    interest_rate = models.DecimalField(max_digits=6, decimal_places=2)
    term_months = models.PositiveIntegerField()

    # El préstamo nace activo: el desembolso ocurre en la misma
    # transacción de aceptación. No existe un estado "pending".
    status = models.CharField(
        max_length=16,
        choices=Status.choices,
        default=Status.ACTIVE,
    )

    class Meta:
        # CONSTRAINTS: reglas que también valida PostgreSQL.
        # Aunque alguien saltee la API, la base no acepta un monto <= 0.
        constraints = [
            models.CheckConstraint(
                condition=models.Q(principal__gt=0),
                name='loans_loan_principal_positive',
            ),
            models.CheckConstraint(
                condition=models.Q(term_months__gt=0),
                name='loans_loan_term_months_positive',
            ),
        ]

    def __str__(self):
        # MÉTODO SIMPLE: solo sirve para logs y para el admin.
```

```
        # No mueve dinero ni dispara operaciones.
        return f'Loan<{self.id}:{self.principal}:{self.status}>'
```

Lo importante es lo que **no** tiene la clase:

- no tiene un `save()` que desembolse dinero;
- no tiene un método `accept()` que llame al ledger;
- no conoce las clases internas de `ledger` ni de `offers`;
- no dispara emails ni notificaciones por sí sola.

Eso está afuera, en un servicio. El modelo sabe **qué es** un préstamo; el servicio sabe **cómo se acepta** un préstamo. Esa separación evita que un guardado accidental mueva plata.

4.4 Las operaciones: servicios como funciones

Las operaciones que escriben son **funciones** en `services.py`, no métodos de un "LoanService" o un "LedgerManager". Otra analogía útil:

> El **modelo** es el expediente. El **servicio** es el cajero que ejecuta el trámite. El **selector** es el archivo donde se consulta. El **serializer** es el formulario que se recibe en la ventanilla. La **vista** es la ventanilla.

El servicio `accept_offer` es el ejemplo más completo. Está comentado paso a paso:

```
def accept_offer(borrower_user_id, offer_id, idempotency_key):
    # 1. Calculamos un hash del pedido para detectar si la misma clave
    #    de idempotencia llega con un payload distinto.
    request_hash = ledger_services.build_request_hash({'offer_id': str(offer_id)})

    with transaction.atomic():
        # 2. Si esta operación ya se hizo con la misma clave, devolvemos
        #    el préstamo original. Un doble clic no crea dos préstamos.
        existing = _resolve_idempotent(borrower_user_id, idempotency_key, request_hash)
        if existing is not None:
            return existing

        # 3. Regla de negocio: el prestatario necesita KYC aprobada.
        _require_kyc(borrower_user_id)

        # 4. La app offers bloquea la oferta con select_for_update y la
        #    marca como aceptada. Dos personas no pueden aceptar la misma.
        offer = offers_services.accept_offer(
            offer_id=offer_id,
            borrower_user_id=borrower_user_id,
        )

        # 5. Salvaguarda de la fase 1: cuenta de respaldo verificada con
        #    saldo declarado suficiente para la primera cuota.
        require_repayment_source(
            user_id=borrower_user_id,
            currency=offer.currency,
            minimum_balance=french_installment(
                offer.amount,
                offer.interest_rate,
                offer.term_months,
            ),
        )

        # 6. Creamos el préstamo con las condiciones de la oferta.
        loan = Loan.objects.create(
            borrower_user_id=borrower_user_id,
            lender_user_id=offer.lender_user_id,
            offer_id=offer.id,
            principal=offer.amount,
            currency=offer.currency,
            interest_rate=offer.interest_rate,
            term_months=offer.term_months,
            status=Loan.Status.ACTIVE,
            idempotency_key=idempotency_key,
            request_hash=request_hash,
        )
```

```
        # 7. Congelamos una copia de las condiciones aceptadas.
        #    Si la oferta cambia después, el préstamo no cambia.
        OfferTermsSnapshot.objects.create(
            loan=loan,
            snapshot_data={...},
            is_immutable=True,
        )

        # 8. Calculamos y guardamos el cronograma francés una sola vez.
        _create_installments(loan)

        # 9. Movemos el dinero: lender -> borrower, a través del ledger.
        #    Si el prestamista no tiene saldo, esta llamada falla.
        _disburse(loan)

        # 10. Si todo salió bien, devolvemos el préstamo. Si cualquier paso
        #     anterior falla, transaction.atomic revierte TODO: no queda
        #     préstamo, ni cuotas, ni snapshot, ni dinero movido.
        return loan
```

¿Por qué funciones y no una clase de servicio?

1. **Transacción explícita.** El `with transaction.atomic()` se ve en el código y abarca exactamente el caso de uso.
2. **Orquestación entre apps.** `accept_offer` coordina `offers`, `loans`, `identity` y `ledger` sin que ninguna app conozca las clases internas de otra.
3. **Testabilidad.** Un test llama a la función con datos concretos; no hay que construir un objeto de servicio con dependencias inyectadas.
4. **Menos estado compartido.** No hay un objeto que pueda quedar con datos viejos entre operaciones.
5. **Menos acoplamiento circular.** Si `Loan` tuviera un método `accept_offer()` que llama al ledger, el modelo dependería de otra app y se rompería la regla de import-linter.

Esto se conoce como un enfoque de **transaction script para los casos de uso + modelo de dominio para los datos**. No es "POO pura", pero es una decisión consciente: el dinero se mueve en procedimientos transaccionales claros, no en métodos escondidos dentro de un modelo.

Importante: usar funciones no significa "no usar POO". Significa que la POO se reserva para las entidades, los permisos, las interfaces y los objetos de valor. Los casos de uso se escriben como procedimientos porque justamente necesitamos ver la transacción completa en un solo lugar.

4.5 Permisos, serializers y vistas: clases de infraestructura

DRF permite declarar permisos como clases que heredan de `BasePermission`. Acá la herencia no modela el negocio: **extiende un contrato del framework**. Por ejemplo:

```
class HasVerifiedBackupAccount(BasePermission):
    """Exige una cuenta de respaldo verificada para aceptar un préstamo."""

    message = 'Vinculá y verificá una cuenta de respaldo antes de aceptar un préstamo.'

    def has_permission(self, request, view):
        # Si la exigencia está apagada, no bloqueamos.
        if not settings.REPAYMENT_SOURCE_REQUIRED:
            return True

        # Si está encendida, el usuario autenticado debe tener al menos
        # una cuenta de respaldo en estado verificado.
        return bool(
            request.user
            and request.user.is_authenticated
            and has_verified_backup_account(user_id=request.user.id)
        )
```

Se usaron para reglas de acceso reutilizables:

- `IsKycApproved`: exige KYC aprobada.
- `KycApprovedForUnsafeMethods`: solo exige KYC en POST/PATCH/PUT/DELETE, no en lecturas.
- `IsInvestor`: exige perfil inversor activo.
- `HasVerifiedBackupAccount`: exige cuenta de respaldo verificada para aceptar un préstamo.
- `IsActiveProfile`: valida que el perfil activo sea el que corresponde a la operación.

Los **serializers** y las **vistas** también son clases de DRF (`ModelSerializer`, `APIView` o vistas genéricas). Su trabajo es recibir y validar datos, no decidir reglas de dinero. La regla también vive en `services.py` (**defense in depth**): el permiso da un 403 temprano y el service vuelve a validar antes de tocar el dinero. Si mañana se llama al service desde un comando o un test, la regla sigue vigente.

4.6 Interfaces y polimorfismo: el caso de los pagos

Este es el lugar donde la POO se usa de forma más clásica. `PaymentProvider` es una clase abstracta (`abc.ABC`) que define el contrato. Pensalo como un **enchufe estándar**: el sistema dice “necesito estos métodos”; cada proveedor decide cómo los implementa.

```
class PaymentProvider(abc.ABC):
    """Contrato que todo proveedor de pagos tiene que cumplir."""

    name: str = ''

    @abc.abstractmethod
    def create_deposit(self, *, payment_id, user_email, amount, currency, ...):
        # El sistema pide "creá un depósito"; el proveedor decide cómo.
        ...

    @abc.abstractmethod
    def create_withdrawal(self, *, payment_id, user_email, amount, currency, ...):
        # Mismo contrato para un retiro.
        ...

    @abc.abstractmethod
    def create_transfer(self, *, payment_id, amount, currency, ...):
        # Transferencia entre usuarios de Presto.
        ...

    @abc.abstractmethod
    def get_balance(self) -> Decimal:
        # Saldo en la cuenta del proveedor.
        ...

    @abc.abstractmethod
    def verify_webhook_signature(self, *, body, headers) -> bool:
        # Valida la firma ANTES de interpretar el cuerpo del webhook.
        ...
```

Hay dos implementaciones:

- `SimulatedProvider`: demo y tests, confirma sin salir a internet.
- `MercadoPagoProvider`: llama a Mercado Pago y valida la firma HMAC del webhook.

El service de pagos no sabe cuál está activo. Recibe la implementación según `PAYMENTS_PROVIDER` y llama siempre al mismo método. Eso es **polimorfismo**: mismo mensaje (`create_deposit`), distinto comportamiento según el objeto. Y visto desde arquitectura es un **puerto con adaptadores**: el dominio define lo que necesita y la infraestructura provee la implementación.

Los resultados también son objetos simples (`dataclasses`):

- `ProviderResult`: `external_id`, `status`, `payload`, `checkout_url`.
- `WebhookEventInfo`: lo que el proveedor entendió de un webhook.

Así se evita pasar diccionarios sueltos por todo el flujo y se mantiene un contrato claro entre capas.

4.7 Lo que se evitó a propósito

- **Modelos gordos con lógica financiera.** Un `Loan.save()` que desembolse dinero sería peligroso: se ejecutaría en cualquier guardado, incluso en un test o en el admin. El desembolso es un caso de uso

explícito.

- **Una capa de repositorios sobre el ORM.** Django ya es el repositorio; agregar otra capa solo duplicaría métodos.
- **Una clase por servicio.** No hay `LoanService`, `LedgerService`, `InvestmentService` con estado. Las funciones son más directas.
- **Herencia múltiple o jerarquías de dominio.** Se prefirió composición: `loans` compone llamadas a `ledger`, `identity` y `offers`.
- **DDD completo con agregados y eventos.** Para el tamaño del proyecto habría sido sobreingeniería. Se tomaron las ideas útiles: lenguaje ubicuo, límites de apps, invariantes explícitos y snapshot de condiciones.

4.8 Cómo defender esta decisión en la exposición

Si surge la pregunta "¿usaron POO?", una respuesta corta y sólida:

> "Sí. Usamos clases para las entidades del dominio, para los permisos y para la interfaz de pagos. Los casos de uso que mueven dinero los escribimos como funciones transaccionales para que la transacción se vea completa en un solo lugar. No queremos que un `save()` accidental desembolse un préstamo."

Y si preguntan por qué no hay `LoanService`:

> "Porque el servicio no tiene estado propio. Necesitamos orquestar varias apps dentro de una transacción atómica y que el flujo se lea de arriba a abajo. Una función expresa mejor ese caso de uso; una clase solo agregaría un objeto que no representa nada del negocio."

5. Las decisiones críticas sobre el dinero

5.1 `Decimal`, nunca `float`

Todos los importes son `DecimalField(max_digits=18, decimal_places=2)`. En Python se usa `Decimal`; en la API viajan como **string** (`"1250.50"`), no como número de punto flotante. En el front se formatean sin convertirlos a `float`.

El redondeo está centralizado en `apps/core/money.py`:

```
def round_two(value: Decimal) -> Decimal:
    return value.quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)
```

Ningún otro módulo redondea. Eso evita diferencias de un centavo entre el cronograma, el ledger y la pantalla.

5.2 Libro mayor de doble entrada e inmutable

El corazón del sistema es `ledger`:

- `Account`: cuenta de usuario (`user_wallet`), custodia (`custody`) o sistema (`system`).
- `LedgerTransaction`: movimiento que agrupa asientos; tiene tipo, descripción, idempotencia y fecha.
- `LedgerEntry`: cada débito o crédito sobre una cuenta.

Invariantes:

1. **Doble entrada:** por cada transacción, la suma de débitos es igual a la suma de créditos.

2. **Inmutabilidad:** triggers de PostgreSQL bloquean `UPDATE` y `DELETE` sobre transacciones y asientos.
3. **Saldo no negativo** en billeteras de usuario; las cuentas de contrapartida pueden quedar negativas porque representan el mundo exterior.
4. **Suma cero por moneda:** la suma de todos los saldos de una moneda es cero.
5. **Corrección por reverso:** un error no se edita; se crea una transacción compensatoria que referencia la original.

El saldo está **materializado** en `Account.balance` para que las consultas sean rápidas, pero se actualiza únicamente dentro de `services.py`, bajo `select_for_update()`. Hay tests que verifican que el saldo coincide con la suma de sus asientos.

¿Por qué triggers en la base y no solo validaciones en Python? Porque una regla contable no debería depender de que todos los caminos de código se acuerden de cumplirla. La base es la última línea de defensa; Python da mensajes claros y validaciones tempranas.

5.3 Idempotencia

Toda operación que mueve dinero recibe un `Idempotency-Key`. El backend guarda:

- la clave;
- un `request_hash` del payload;
- la transacción original.

Si llega la misma clave con el mismo payload, devuelve la operación original. Si llega la misma clave con otro payload, responde `409`. Esto evita duplicar un depósito por un doble clic, un reintento de red o un webhook repetido.

En el frontend, `useIdempotencyKey` genera una clave por intento del usuario y la reutiliza en los reintentos. Se resetea después de un éxito.

5.4 Atomicidad y concurrencia

Las operaciones sensibles usan:

- `transaction.atomic()`: todo o nada;
- `select_for_update()`: bloqueo de filas para evitar carreras;
- orden ascendente de bloqueo por `id` en las cuentas: evita deadlocks;
- recuperación de carrera de idempotencia: si dos requests simultáneos usan la misma clave, uno gana y el otro recupera la operación original.

Ejemplo: aceptar una oferta bloquea la oferta, valida saldo y condiciones, crea el préstamo, el snapshot y las cuotas, y recién entonces desembolsa. Si el desembolso falla, **todo** se revierte y la oferta vuelve a estar disponible.

5.5 Snapshot de condiciones

`OfferTermsSnapshot` guarda una copia JSON de la oferta aceptada: monto, moneda, tasa, plazo, requisitos, prestamista, prestatario y fecha. Es inmutable (`is_immutable=True`).

¿Por qué un JSON y no columnas normalizadas? Porque el snapshot es evidencia de lo que se aceptó, no una entidad que se vaya a consultar y actualizar. Si mañana cambia el modelo de ofertas, el snapshot histórico sigue siendo legible. La UI muestra los campos principales y deja el JSON detrás de “Ver datos técnicos”.

¿Por qué no copiar solo la oferta actual? Porque la oferta puede editarse o eliminarse. El préstamo debe conservar las condiciones con las que nació. Es la diferencia entre "la oferta de hoy" y "el contrato que acepté".

5.6 Cronograma francés y última cuota

`build_installment_schedule` calcula la cuota francesa:

```
cuota = P * i / (1 - (1 + i)^-n)
```

donde `i` es la tasa mensual equivalente a la TNA y `n` la cantidad de meses. No se redondea en pasos intermedios: se redondea una vez por cuota. La **última cuota ajusta el capital** para que la suma de capital sea exactamente el principal y no queden centavos sueltos.

Para plazos fijos se usa interés simple:

```
interés = monto * TNA / 100 * días / 365
```

La TEA y el costo total se calculan en `core/money.py` para mostrarlos antes de aceptar.

5.7 Máquinas de estado

- `Loan`: `active → paid`, `active → defaulted`, `defaulted → active`, `defaulted → paid`. Las transiciones inválidas lanzan error de dominio.
- `Installment`: `pending → paid`, `pending → overdue`, `overdue → paid`.
- `Payment`: `pending → confirmed/failed`, `confirmed → reversed`.
- `Position`: `active → matured`.

No se usa un estado `pending` en `Loan`: el préstamo nace `active` porque el desembolso ocurre en la misma transacción de aceptación. Si el desembolso falla, no hay préstamo.

---

6. Modelo de datos en una vista rápida

App	Entidad	Se relaciona con	Nota
identity	`User`	—	Email único, sin `username`
identity	`PrestoProfile`	`User`	Perfil operativo activo
identity	`KYCVerification`	`User`	Historial de verificaciones
identity	`BackupAccount`	`User`	Cuenta de respaldo declarada
ledger	`Account`	`user_id` (UUID)	Billetera, custodia o sistema
ledger	`LedgerTransaction`	`created_by_user_id`	Movimiento con idempotencia
ledger	`LedgerEntry`	`Account`, `LedgerTransaction`	Débito o crédito
payments	`Payment`	`user_id`	Intención de pago
payments	`WebhookEvent`	`payment_id`	Evento externo único
offers	`Offer`	`lender_user_id`	Oferta activa, expirada o aceptada
loans	`Loan`	`borrower_user_id`, `lender_user_id`, `offer_id`	Préstamo aceptado
loans	`OfferTermsSnapshot`	`Loan`	Condiciones inmutables

App	Entidad	Se relaciona con	Nota
loans	`Installment`	`Loan`	Cuotas ordenadas
investments	`InvestmentProduct`	—	Producto de plazo fijo
investments	`Position`	`user_id` , `InvestmentProduct`	Inversión abierta
budgets	`Category`	`user_id`	Categoría de gasto
budgets	`Expense`	`Category` , `ledger_transaction_id`	Clasifica, no guarda importe
budgets	`Budget`	`Category`	Límite mensual por moneda
notifications	`Notification`	`user_id`	Aviso y estado de envío
notifications	`NotificationPreference`	`user_id`	Opt-out por evento

La decisión de usar UUID en las referencias cruzadas tiene una consecuencia práctica: el frontend y los tests pueden construir IDs sin depender de una secuencia numérica, y exponer un ID no revela cuántos usuarios u operaciones hay.

---

7. Diseño de la API

- **REST versionada** bajo `/api/v1/` .
- **JSON** con UUID públicos, fechas ISO 8601 UTC e importes como string.
- **Errores uniformes:**

```
{
  "error": {
    "code": "insufficient_funds",
    "message": "Saldo insuficiente.",
    "details": {}
  }
}
```

- **Paginación estándar** (`count` , `next` , `previous` , `results`) y paginación por cursor para movimientos del ledger.
- **OpenAPI** generado con drf-spectacular. El esquema se versiona en `frontend/contract/openapi.yaml` y el frontend genera sus tipos con `pnpm gen:api` . Así, si la API cambia, TypeScript falla antes de producción.
- **Throttling** por scope: login, registro, reseteo, verificación, webhooks y asistente.
- **Cabecera** `Idempotency-Key` en operaciones de dinero.
- **Cabecera** `X-Presto-Mode: demo` cuando `DEMO_MODE=true` , para que el front muestre el aviso correspondiente.

7.1 Autenticación

- El **access token** vive solo en memoria del frontend.
- El **refresh token** viaja en una cookie `httpOnly` , `Secure` y `SameSite` configurable (`None` en producción porque front y API están en dominios distintos).
- El refresh rota y el token anterior queda en blacklist.
- Ante un `401` , el cliente HTTP intenta refrescar una vez y repite la petición.

- Logout invalida el refresh.

¿Por qué no guardar el access token en `localStorage`? Porque sería accesible desde JavaScript ante un XSS. En memoria se pierde al recargar, pero se recupera con el refresh cookie, que JavaScript no puede leer.

---

8. Frontend: decisiones de estructura

8.1 React funcional, no clases

El frontend usa **componentes funcionales y hooks**. No hay componentes de clase. La lógica de sesión, tema y sidebar vive en contextos:

- `AuthContext`: usuario, login, logout, refresh, perfil activo.
- `ThemeContext`: tema claro/oscuro, persistencia y `prefers-color-scheme`.
- `SidebarContext`: estado del menú en desktop y mobile.

No se agregó Redux ni otra librería de estado global porque el estado compartido es acotado y los datos del servidor se piden por pantalla. Agregar un store global habría sumado complejidad sin resolver un problema real.

8.2 Organización por features

```
frontend/src/
  app/            layout, rutas, providers, header, sidebar
  features/
    auth/         login, registro, perfil, KYC, contexto de sesión
    wallet/       dashboard, billetera
    payments/     depósitos, retiros, transferencias
    marketplace/  búsqueda y comparación de ofertas
    loans/        préstamos y detalle de cuotas
    investments/  productos y posiciones
    budgets/      categorías y presupuestos
    notifications/avisos
    assistant/    Presti
    marketing/    landing y legales
  shared/
    api/          cliente HTTP, endpoints, tipos generados
    lib/          formato de dinero/fechas, helpers
    ui/           iconos, mascota, toggle de tema
```

Cada feature tiene sus páginas y componentes. Lo transversal vive en `shared` y `app`. Esto evita un `components/` gigante sin relación y hace que un cambio de préstamos no obligue a recorrer todo el proyecto.

8.3 Cliente HTTP centralizado

`shared/api/client.ts` concentra:

- URL base por variable de entorno;
- `credentials: include` para la cookie de refresh;

- extracción del mensaje de error del backend;
- reintento automático ante `401`;
- `Idempotency-Key` en operaciones de dinero;
- normalización de respuestas paginadas (`asArray`, `asPage`).

Las pantallas no llaman a `fetch` directamente. Eso permite cambiar la autenticación o el manejo de errores en un solo lugar.

8.4 Tipos generados desde OpenAPI

`src/shared/api/schema.d.ts` se genera desde el contrato. No se edita a mano. Los tipos de `LoanDetail`, `Offer`, `Installment`, etc. salen del esquema real. La ventaja es que un cambio incompatible en el backend se detecta en `pnpm typecheck`, no en la demo.

8.5 Diseño visual y modo oscuro

- Tailwind CSS 4 con clases semánticas (`.card`, `.btn`, `.badge`, `.alert`, `.stat-card`) para no repetir utilidades.
- Variables y estilos propios en `index.css`.
- Dark mode con clase `.dark` en `<html>`, persistencia en `localStorage` (`presto-theme`) y respeto de `prefers-color-scheme`.
- Script inline en `index.html` para aplicar el tema antes de renderizar y evitar el flash blanco.
- Toggle reutilizable (`ThemeToggle`) en header, landing, login, registro y legales.
- Responsive mobile-first; el menú se convierte en drawer y las tablas se envuelven en contenedores con scroll.

La decisión de usar CSS propio + Tailwind, en lugar de una librería de componentes completa, fue mantener control visual y evitar una dependencia grande para una demo con identidad propia.

---

9. Pruebas y calidad

9.1 Backend

- **899 tests** con pytest, contra PostgreSQL real (no SQLite).
- `factory_boy` para construir usuarios, cuentas, ofertas, préstamos y posiciones.
- Tests de servicios, API, permisos, concurrencia, idempotencia, triggers y flujo end-to-end.
- El test end-to-end recorre el circuito completo en ARS y USD y verifica el invariante de suma cero del ledger.
- Cobertura global mínima del 80%, con gates más altos para `budgets`, `notifications` y `reports`.

Los tests se enfocan en **invariantes**, no en detalles de implementación:

- el saldo coincide con los asientos;
- no se puede gastar más de lo que hay;
- no se puede pagar una cuota fuera de orden;
- aceptar dos veces la misma oferta no crea dos préstamos;
- un tercero no puede ver un préstamo ajeno;
- un préstamo sin cuenta de respaldo verificada no se acepta.

9.2 Calidad estática

- **ruff** para lint y formato (línea 100, comillas simples).
- **import-linter** con 32 contratos que verifican los límites entre apps.
- `makemigrations --check` para que no falten migraciones.
- `spectacular --validate --fail-on-warn` para validar el OpenAPI.
- `pip-audit` para dependencias con vulnerabilidades conocidas.
- `check --deploy` para la configuración de producción.

9.3 Frontend

- **37 tests unitarios** con Vitest: formato de dinero, token en memoria, refresh, idempotencia y banners.
- `pnpm typecheck`, `pnpm lint` y `pnpm build` en CI.
- Specs de Playwright para flujos E2E; se ejecutan manualmente y **no** están en CI en esta versión.

9.4 CI

GitHub Actions corre 5 jobs:

1. calidad (ruff + import-linter + OpenAPI + auditoría + deploy check);
2. migraciones;
3. tests contra PostgreSQL 16;
4. frontend (typecheck + tests + build);
5. imagen Docker (build, verificación y publicación en GHCR en `main`).

---

10. Seguridad, privacidad y modo demo

10.1 Seguridad de la aplicación

- Contraseñas con **Argon2** y validadores de Django.
- JWT con access corto, refresh rotativo y blacklist.
- Refresh en cookie `httpOnly`, no accesible desde JavaScript.
- Throttling en login, registro, reseteo, verificación, webhooks y asistente.
- CORS explícito con credenciales, sin `*`.
- Producción con `DEBUG=False`, HTTPS, HSTS, cookies seguras y `ALLOWED_HOSTS` explícito.
- Contenedores con usuario sin privilegios (`presto`, uid 1000).
- Secretos solo por variables de entorno; `.env` fuera del repositorio.

10.2 Modo demo

`DEMO_MODE=true` habilita:

- depósitos y retiros simulados;
- KYC automático o aprobado desde el admin;
- cabecera `X-Presto-Mode: demo`;
- aviso visible en el front;
- proveedor de pagos simulado.

Pero **no** habilita dinero real: no hay pasarela conectada, no se mueven fondos y los saldos no son exigibles. El backend tiene un system check (`ledger.E001`) que bloquea depósitos simulados en producción si `DEMO_MODE` no está explícitamente activo.

10.3 KYC, biometría y cuentas de respaldo

- El KYC es una simulación administrativa: se aprueba desde el admin, no valida documentos reales.
- La validación biométrica es simulada y determinista; guarda un hash de referencia, nunca la imagen ni datos biométricos reales.
- `BackupAccount` guarda proveedor, referencia enmascarada, moneda y saldo declarado. No guarda credenciales, tokens ni números de cuenta completos.
- En demo, la verificación de la cuenta de respaldo usa un código fijo (`123456`). En una integración real habría que verificar titularidad y saldo contra el proveedor.

10.4 Salvaguardas de la fase 1

Para reducir el riesgo de impago en la demo se implementó:

1. **Cuenta de respaldo verificada obligatoria** para aceptar un préstamo, con saldo declarado suficiente para la primera cuota.
2. **Bloqueo de desvinculación** de la cuenta de respaldo si el usuario tiene un préstamo activo o en mora.
3. **Bloqueo de cierre de cuenta** si hay préstamos activos o inversiones abiertas.
4. **Baja lógica** de la cuenta (`is_active=False`), no borrado físico: el ledger es inmutable y se conserva para auditoría.

10.5 Qué haría falta para una integración real

La demo **no** debita de un banco ni de Binance. Una integración real requiere, como mínimo:

- contrato y credenciales del proveedor (banco, billetera o exchange);
- consentimiento explícito del usuario y gestión de revocación;
- webhooks firmados y reconciliación de estados;
- verificación de titularidad y saldo;
- cumplimiento de KYC/PLA-FT, BCRA, CNV y protección de datos;
- auditoría, monitoreo y planes de contingencia.

Ese encuadre está documentado en `regulatorio-bcra.md` y en los ADR de pagos y custodia. Presti también lo explica dentro de la app.

---

11. Infraestructura y despliegue

11.1 Docker

El `Dockerfile` es multi-stage:

- `base` : Python 3.14 slim, usuario `presto` , directorio `/app` .
- `runtime` : dependencias de producción, código copiado después de instalar dependencias para aprovechar la caché, `collectstatic` .
- `development` : agrega pytest, ruff, import-linter, factory-boy y pip-audit.

- `production`: etapa final usada por Render o `docker build`.

`deploy/entrypoint.sh` aplica migraciones con reintentos si `RUN_MIGRATIONS=1` y luego arranca Gunicorn con 2 workers.

11.2 Render y Vercel

- **API en Render:** blueprint `render.yaml` con PostgreSQL, Redis y el web. El web usa una imagen preconstruida de GHCR; no compila en Render.
- **Front en Vercel:** root directory `frontend`, variables `VITE_API_URL` y `VITE_ENABLE_ASSISTANT`.
- **CI/CD:** push a `main` → GitHub Actions → imagen en GHCR → Render la toma → Vercel despliega el front.

11.3 Límites conocidos del plan free

- Render free **no ofrece background workers**: no corren Celery worker ni beat. Por eso los emails y las tareas programadas (vencimientos, liquidaciones) no se ejecutan automáticamente en la demo pública.
- El web service **se duerme** por inactividad; la primera visita puede tardar 30–60 segundos.
- La base PostgreSQL free **expira a los 30 días** de creada.
- No hay Sentry ni backups automáticos probados en esta instancia.

Estos límites son de infraestructura, no del código: con un plan pago se agregan worker y beat usando la misma imagen y los comandos documentados en `render.yaml`.

---

12. Trade-offs: qué se ganó y qué se dejó pendiente

Decisión	Beneficio	Costo	Cuándo revisar
Monolito modular	Menos complejidad operativa, transacciones simples	Un solo despliegue y un solo equipo	Si una app necesita escalar o desplegarse por separado
Servicios como funciones	Casos de uso explícitos y atómicos	Menos "POO clásica" en las reglas	Si aparecen muchos servicios con estado compartido
UUID sin FK entre apps	Desacople y extracción futura	Integridad referencial no la garantiza la base	Si aumentan las referencias cruzadas y los errores
Snapshot JSON	Evidencia histórica flexible	No se consulta por campos del snapshot	Si se necesitan reportes sobre condiciones históricas
Saldo materializado	Consultas rápidas	Hay que mantenerlo sincronizado	Si el volumen exige otra estrategia de lectura
Triggers en PostgreSQL	Invariantes fuertes e independientes del código	Menos portabilidad de la base	Si se cambia de motor (poco probable)
Access token en memoria	Menor exposición a XSS	Se pierde al recargar; depende del refresh cookie	Si se necesita sesión offline o multi-tab más compleja
Sin LLM en Presti	Respuestas deterministas, sin alucinaciones ni costo por token	Menos flexibilidad conversacional	Si se agrega un LLM con herramientas y validación
Worker en plan pago	Automatización real de emails y vencimientos	Costo mensual	Cuando la demo pase a piloto
Productos de inversión desde admin	Control y trazabilidad	No hay autoservicio para crearlos	Si se necesita un backoffice propio

Decisión	Beneficio	Costo	Cuándo revisar
Perfiles fijos	Reglas claras para la demo	No se puede cambiar de rol	Si el producto real permite multi-perfil

Deuda técnica consciente

- No hay integración real de pagos ni de cuentas bancarias.
- El KYC y la biometría son simulados.
- Los emails en la demo salen por consola si no hay SMTP.
- Los E2E de Playwright no corren en CI.
- No hay observabilidad avanzada (Sentry, métricas, alertas).
- El cierre de cuenta es lógico; no hay borrado físico por la inmutabilidad del ledger.
- La página legal es un borrador de alcance, no términos definitivos.

---

13. Mapa rápido del código

Qué querés mostrar	Dónde está
Modelo base y utilidades de dinero	`apps/core/models.py`, `apps/core/money.py`
Usuarios, perfiles, KYC y respaldo	`apps/identity/models.py`, `apps/identity/services.py`
Ledger, idempotencia y bloqueos	`apps/ledger/models.py`, `apps/ledger/services.py`
Pagos y proveedores	`apps/payments/providers.py`, `apps/payments/services.py`
Ofertas y matching	`apps/offers/`, `apps/matching/`
Préstamos, snapshot y cuotas	`apps/loans/models.py`, `apps/loans/services.py`
Inversiones	`apps/investments/services.py`
Presupuestos derivados del ledger	`apps/budgets/selectors.py`
Notificaciones	`apps/notifications/receivers.py`, `apps/notifications/tasks.py`
Presti	`apps/assistant/services.py`, `apps/assistant/regulations.py`
Rutas y protección del front	`frontend/src/app/App.tsx`
Sesión y cliente HTTP	`frontend/src/features/auth/AuthContext.tsx`, `frontend/src/shared/api/client.ts`
Tema oscuro	`frontend/src/app/providers/ThemeContext.tsx`, `frontend/src/index.css`
Infra	`Dockerfile`, `render.yaml`, `frontend/vercel.json`, `.github/workflows/ci.yml`

Comandos útiles

```
pytest -q
ruff check . && ruff format --check .
lint-imports
python manage.py makemigrations --check --dry-run
python manage.py spectacular --validate --fail-on-warn --file /tmp/schema.yml
python manage.py seed_demo
python manage.py check --deploy
```

En Windows, con el entorno del proyecto:

```
. .\scripts\env.ps1
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe manage.py runserver
```

14. Cómo explicarlo en dos minutos

Si en la instancia técnica hay poco tiempo, este es el hilo:

1. **"Es un monolito modular en Django, no microservicios."** Un solo despliegue y una sola base, con apps separadas por responsabilidad y 32 contratos de import-linter que impiden que se mezclen.
2. **"Las reglas de negocio viven en servicios, no en las vistas."** Las vistas validan y delegan; los servicios son transaccionales; los selectores leen; los modelos son entidades con constraints.
3. **"Usamos POO donde aporta: modelos, permisos, serializers e interfaces de pago."** Los casos de uso son funciones para que la transacción sea explícita y no quede escondida en un `save()`.
4. **"El dinero pasa por un ledger de doble entrada inmutable."** Débitos y créditos, triggers en PostgreSQL, idempotencia y reversos. El saldo se materializa, pero la fuente de verdad son los asientos.
5. **"El préstamo congela sus condiciones en un snapshot."** Si la oferta cambia o se elimina, el préstamo mantiene lo aceptado. Las cuotas se calculan una vez con amortización francesa.
6. **"El front es React + TypeScript con tipos generados desde OpenAPI."** El access token vive en memoria, el refresh en cookie httpOnly, y el cliente centraliza errores, refresh e idempotencia.
7. **"La demo es honesta: dinero simulado, KYC simulado y límites documentados."** Las salvaguardas de la fase 1 están implementadas; la integración real requiere contratos, consentimiento y encuadre regulatorio.

15. Glosario mínimo

Término	Significado en Presto
Ledger	Libro mayor contable donde se registran débitos y créditos
Asiento (`LedgerEntry`)	Cada débito o crédito de una transacción
Doble entrada	Todo movimiento tiene débitos y créditos que suman igual
Idempotencia	Repetir la misma operación no la duplica
Snapshot	Copia congelada de las condiciones de una oferta aceptada
TNA	Tasa nominal anual, expresada en porcentaje
TEA	Tasa efectiva anual equivalente
Amortización francesa	Cuota fija con interés decreciente y capital creciente
KYC	Verificación de identidad del usuario
PLA/FT	Prevención de lavado de activos y financiamiento del terrorismo
Monolito modular	Un solo despliegue, separado internamente en módulos
Selector	Función de lectura que consulta datos sin modificarlos
Service	Función de escritura que aplica reglas y transacciones

Cierre. La arquitectura de Presto busca que la demo se vea simple por fuera y sea consistente por dentro. La decisión central no fue “usar muchas tecnologías”, sino **poner las reglas del dinero en un lugar explícito, transaccional y auditable**, y dejar el resto del código lo más claro posible para poder explicarlo, probarlo y evolucionarlo.